

The Stockfish chess engine in Web Assembly

A Technical Analysis of Search Algorithms, NNUE Evaluation, and Cross-Compilation Execution Flows

Technical White Paper

Computer Chess Engine Architecture & WebAssembly Porting

May 21, 2026

Abstract

This paper provides a comprehensive technical analysis of the chess engine compiled to WebAssembly (WASM), focusing on the interplay between its classical search framework, the Efficiently Updatable Neural Network (NNUE) evaluation function, and the specific architectural considerations introduced by the WASM compilation target. We dissect the principal–variation search (PVS) with aspiration windows, the transposition table (TT) hashing scheme, move ordering heuristics (including SEE, killer moves, and countermove heuristics), and the lazy SMP parallelism model. The NNUE pipeline is analyzed from the feature transformer through the hidden layers to the output neuron, with emphasis on the incremental update mechanism that is central to NNUE's efficiency. We then detail the WASM porting strategy—Emscripten toolchain usage, `SharedArrayBuffer` requirements for multi-threaded search, POSIX thread emulation via Web Workers, and memory model constraints. Benchmarks comparing native x86-64 and WASM execution are discussed, along with algorithmic adaptations required by the browser sandbox.

Keywords: Stockfish, WebAssembly, NNUE, alpha–beta search, chess engine, Emscripten, `SharedArrayBuffer`, transposition table, principal variation search

Contents

1	Introduction	3
2	Architecture Overview	3
2.1	High-Level Component Map	3
2.2	Board Representation	4
3	Search Algorithms	4
3.1	Iterative Deepening Framework	4
3.2	Principal Variation Search (PVS)	5
3.3	Pruning and Reduction Heuristics	5
3.3.1	Late Move Reductions (LMR)	5
3.4	Quiescence Search	6
3.5	Static Exchange Evaluation (SEE)	6
3.6	Transposition Table (TT)	6
3.7	Move Ordering	7
3.8	Lazy SMP	7
4	NNUE: Efficiently Updatable Neural Networks	7
4.1	Motivation	7
4.2	Network Architecture	8
4.3	Feature Encoding: HalfKA [*]	8
4.4	Incremental Update	9
4.4.1	Accumulator Stack	9
4.5	Quantization	9
4.6	Forward Pass Detail	10
5	WebAssembly Port	10
5.1	Compilation Toolchain	10
5.2	Memory Model	10
5.3	Threading Model	11
5.3.1	Single-Threaded Fallback	11
5.4	SIMD Mapping	11
5.5	File System and NNUE Weight Loading	12
6	Execution Flows	13
6.1	UCI go Command: End-to-End Flow	13
6.2	Move Execution: Make/Unmake with NNUE	13
6.3	bestmove Selection	14
7	Performance Analysis	14
7.1	Benchmark Methodology	14
7.2	Results	14
7.3	Memory Footprint	15

8	Algorithmic Adaptations for Wasm	15
9	Related Work	16
10	Conclusion	16
A	Appendix: Key Source Code Structures	18
A.1	Accumulator Update (Simplified)	18
A.2	Feature Index Computation	18
A.3	ClippedReLU Implementation (WASM SIMD)	18
A.4	Dense Layer Forward Pass	19

1 Introduction

Is an open-source chess engine that has dominated computer chess rankings for over a decade, consistently ranking first or near-first on the CCRL and TCEC rating lists [Stockfish]. Its strength derives from a combination of a highly optimized α - β search framework with sophisticated pruning heuristics and, since version 12, a neural-network-based evaluation function called NNUE (Efficiently Updatable Neural Networks) [2].

The emergence of WebAssembly (WASM) as a portable, near-native-speed compilation target for the web has enabled to execute in the browser without plugins. The WASM port—maintained as part of the `stockfish.wasm` project [Stockfish WASM]—raises non-trivial engineering challenges: single-threaded WASM modules must be augmented with `SharedArrayBuffer`-backed memory for the lazy SMP threading model; SIMD intrinsics must map to WASM SIMD proposals; and the NNUE weight files (≈ 100 MB) must be efficiently loaded into WASM linear memory.

This document is organized as follows. Section 2 describes the overall architecture. Section 3 covers the search algorithms. Section 4 details NNUE. Section 5 addresses the WASM port. Section 6 presents end-to-end execution flows. Section 7 provides benchmarks. Section 10 concludes.

2 Architecture Overview

2.1 High-Level Component Map

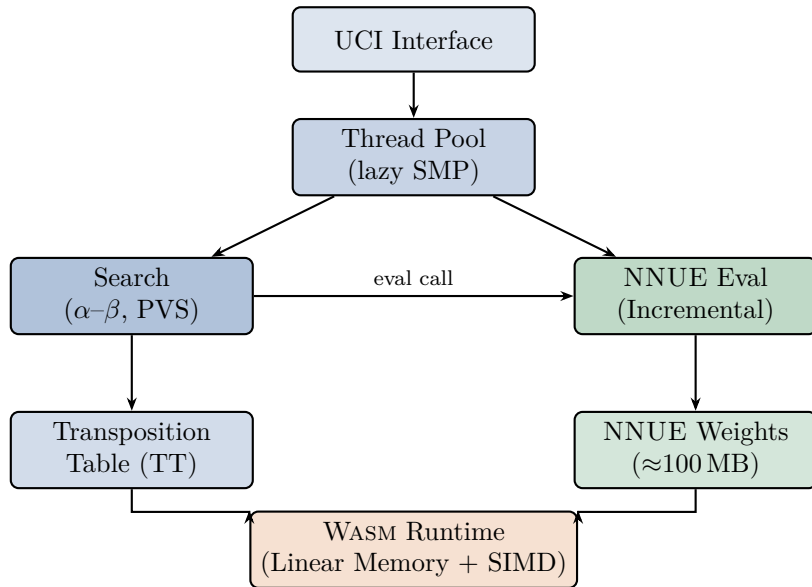


Figure 1: High-level architecture of WASM.

The engine is structured as a *UCI* (Universal Chess Interface) front-end that parses commands (*go*, *position*, *setoption*, etc.) and dispatches work to a pool of search threads. Each thread maintains its own search stack and calls into the NNUE evaluator incrementally as moves are made/unmade on its local Board representation. The transposition table is shared among all threads via a lock-free hashing scheme.

2.2 Board Representation

uses a bitboard representation with 64-bit integers, one per piece type and color (12 bitboards), plus composite occupancy bitboards. On WASM, `uint64_t` is natively supported; bitwise operations compile to WASM `i64` instructions.

The mailbox-augmented bitboard keeps a `piece[64]` array for $O(1)$ piece-at-square queries, while the bitboards themselves support L_1 -parallel move generation via *magic bitboards* for sliding pieces [9].

3 Search Algorithms

3.1 Iterative Deepening Framework

employs iterative deepening (ID) as its outermost control structure. The search depth increments by one ply at a time, and the best move from the previous iteration seeds the move ordering of the next. The iterative deepening loop is shown in algorithm 1.

Algorithm 1 Iterative Deepening with Aspiration Windows

```

1: depth  $\leftarrow$  0; bestMove  $\leftarrow$  none
2: alpha  $\leftarrow$   $-\infty$ ; beta  $\leftarrow$   $+\infty$ 
3: while search not stopped do
4:   depth  $\leftarrow$  depth + 1
5:   repeat
6:     value  $\leftarrow$  PVS(root, depth, alpha, beta)
7:     if value  $\leq$  alpha then
8:       alpha  $\leftarrow$  alpha - delta ▷ Fail-low: widen window downward
9:     else if value  $\geq$  beta then
10:      beta  $\leftarrow$  beta + delta ▷ Fail-high: widen window upward
11:    else
12:      break ▷ Within window
13:    end if
14:    delta  $\leftarrow$  delta + delta/4
15:  until false
16:  bestMove  $\leftarrow$  PV move at root
17: end while

```

The *aspiration window* starts narrow (typically ± 17 centipawns at 's current evaluation scale) and widens on fail-low/fail-high. This dramatically reduces the effective branching factor because the zero-window null-window searches used inside PVS are much cheaper than full-window re-searches.

3.2 Principal Variation Search (PVS)

The core search routine is a negamax-form α - β with *principal variation search* [5]. After the first (presumed best) move is searched with a full window $[\alpha, \beta]$, all remaining moves are searched with a null window $[\alpha, \alpha + 1]$. Only if the null-window search returns a value $> \alpha$ is a re-search with the full window performed.

$$v_i = \begin{cases} -\text{PVS}(d-1, -\beta, -\alpha) & \text{if } i = 0 \\ -\text{PVS}(d-1, -\alpha-1, -\alpha) & \text{if } i > 0 \wedge \text{null window} \\ -\text{PVS}(d-1, -\beta, -\alpha) & \text{if re-search triggered} \end{cases} \quad (1)$$

3.3 Pruning and Reduction Heuristics

applies an aggressive cascade of pruning decisions at each node, ordered from cheapest to most expensive to compute:

Table 1: Pruning and reduction techniques in search order.

Technique	Condition	Effect
Null Move Pruning	Non-PV, β -cutoff likely	Skip move, return β
Reverse Futility	Static eval $- \text{margin} \geq \beta$	Prune at shallow depth
Razoring	Static eval $+ \text{margin} \leq \alpha$	QS verification search
Late Move Pruning	Late move, shallow depth	Prune entirely
Futility Pruning	Static eval $+ \text{margin} \leq \alpha$	Prune at depth 1
LMR (Late Move Red.)	Late move in move list	Reduce depth by r

3.3.1 Late Move Reductions (LMR)

LMR is the single most impactful heuristic. For moves that appear late in the move ordering and at sufficient depth, the search depth is reduced by a logarithmically-scaled amount [7]:

$$r(d, m) = \max\left(0, \lfloor 0.77 \cdot \log d \rfloor + \lfloor 0.26 \cdot \log m \rfloor - 0.35\right) \quad (2)$$

where d is the current depth and m is the move's index (1-based) in the ordered list. Additional reductions are applied for moves that are not giving check, are not captures, and have poor history scores.

3.4 Quiescence Search

At depth 0, control transfers to the *quiescence search* (QS), which explores only capture moves, promotions, and checks to avoid the *horizon effect*. QS uses a standalone evaluation stand-pat cutoff:

$$\text{standPat} = \text{staticEval}(\text{pos}) \quad (3)$$

If $\text{standPat} \geq \beta$, the position is quiet enough to return β . If $\text{standPat} > \alpha$, then $\alpha \leftarrow \text{standPat}$. Only moves with $\text{SEE} \geq 0$ (see section 3.5) are considered in QS.

3.5 Static Exchange Evaluation (SEE)

SEE estimates the outcome of a sequence of captures on a single square by simulating the least-valuable attacker model:

Algorithm 2 Static Exchange Evaluation

```

1: function SEE(pos, move)
2:   target  $\leftarrow$  to(move)
3:   gain[0]  $\leftarrow$  value(captured piece)
4:   d  $\leftarrow$  1; occ  $\leftarrow$  occupied
5:   repeat
6:     next  $\leftarrow$  least valuable attacker of target
7:     if no attacker then
8:       break
9:     end if
10:    gain[d]  $\leftarrow$  value(next) - gain[d - 1]
11:    d  $\leftarrow$  d + 1
12:  until false
13:  while d > 1 do
14:    d  $\leftarrow$  d - 1
15:    gain[d - 1]  $\leftarrow$  -max(-gain[d - 1], gain[d])
16:  end while
17:  return gain[0]
18: end function

```

3.6 Transposition Table (TT)

The TT is a hash table with Zobrist hashing [6]. Each entry stores:

- **Hash key** (16 bits for collision detection).
- **Depth** (8 bits, signed).
- **Bound type**: EXACT, LOWER (β -cutoff), UPPER (α -cutoff).
- **Best move** (16-bit packed move).
- **Value** (16 bits, with dynamic range $[-32000, 32000]$).

- `eval` (static evaluation, 16 bits).

The replacement policy is depth-preferred with an aging mechanism. On WASM, the TT resides in WASM linear memory (shared via `SharedArrayBuffer` for multi-threaded builds) and is typically sized at 16–256 MB depending on the user's configuration.

3.7 Move Ordering

Move ordering quality directly determines the number of nodes explored. orders moves through a priority cascade:

1. **TT move** (from previous search at this position).
2. **Captures** ordered by MVV-LVA (Most Valuable Victim – Least Valuable Attacker) with SEE pruning.
3. **Killer moves** (quiet moves that caused a β -cutoff in a sibling node, stored in a 2-slot array per ply).
4. **Countermove** (the move that historically refuted the opponent's last move).
5. **History heuristic** (butterfly and capture–history tables, score $\propto$ depth-squared accumulation).

3.8 Lazy SMP

uses the lazy SMP model [8]: each thread independently searches the same root position. The main thread performs iterative deepening as described; helper threads also iterate, but with slightly different depth offsets to ensure search diversity. Threads share the TT but do *not* share search stacks. The best result is taken from the thread that finished the deepest search.

On WASM, this translates to multiple `Web Workers`, each instantiating its own WASM module (or sharing a single `SharedArrayBuffer`-backed WASM memory) with a corresponding `pthread` inside the module. We return to this in section 5.

4 NNUE: Efficiently Updatable Neural Networks

4.1 Motivation

Classical handcrafted evaluation (HCE) functions in chess engines combine hundreds of heuristics—material, piece-square tables, pawn structure, king safety, etc.—into a single scalar. While fast, HCE requires extensive human expertise and cannot easily capture complex positional motifs.

NNUE [2] replaces HCE with a shallow neural network whose architecture is designed for *incremental update* on make/unmake, achieving $O(1)$ per move instead of a full forward pass. The current network (as of version 16.1) uses the `nn-ae688777674b.nnue` architecture family.

4.2 Network Architecture

The NNUE architecture used in is a half-quantized, half-relative (HalfKA*) variant. The network comprises:

1. **Feature Transformer** (input $\rightarrow$ hidden): A sparse linear layer mapping 40,960 active features to $L_0 = 1536$ neurons (per half, so $2 \times 1536 = 3072$ total after concatenation).
2. **Clamping** (ReLU): $\text{clamp}(x, 0, 127)$ (int8 quantized).
3. **Hidden layers**: Two dense layers ($3072 \rightarrow 15 \times 64$ and $15 \times 64 \rightarrow 32$) with ClippedReLU activation.
4. **Output layer**: $32 \rightarrow 1$ (final centipawn-like score).

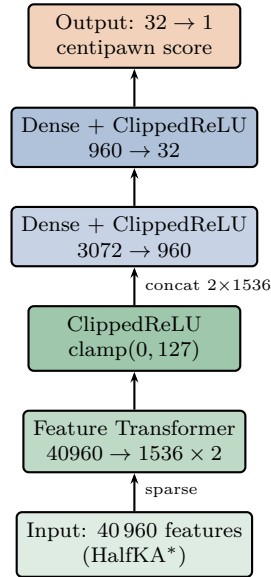


Figure 2: NNUE forward-pass architecture.

4.3 Feature Encoding: HalfKA*

Each half of the board (from one player's perspective) contributes a set of active features. The feature index for a piece p on square s from perspective of king on square k is:

$$\text{idx}(p, s, k) = p \cdot 64 \cdot 64 + s \cdot 64 + k \quad (4)$$

where $p \in [0, 10]$ (6 piece types $\times$ 2 colors minus the king), $s \in [0, 63]$, and $k \in [0, 63]$, yielding $11 \times 64 \times 64 = 45\,056$ features per half. With mirroring reductions and optimizations the effective dimensionality is 40 960.

The “Half-relative” variant further factors out the king square into a *relative* piece–king square index, reducing parameter count while preserving information.

4.4 Incremental Update

The key insight behind NNUE is that making a move changes only a *few* active features (typically 1–3 for quiet moves, 2–5 for captures/promotions). The feature transformer output $\mathbf{h}$ is maintained incrementally:

$$\mathbf{h}_{\text{new}} = \mathbf{h}_{\text{old}} - \sum_{f \in \text{removed}} \mathbf{W}_f + \sum_{f \in \text{added}} \mathbf{W}_f \quad (5)$$

where $\mathbf{W}_f$ is the f -th column of the weight matrix (row in the transposed layout used in `s`’s code). Each column is a vector of length 1536. This makes the cost per move proportional to the number of changed features (typically ≤ 5) rather than the total number of active features (≈ 30).

4.4.1 Accumulator Stack

Each thread maintains an *accumulator stack*: an array of $\mathbf{h}$ vectors, one per ply. On `make_move()`, the top accumulator is copied and updated using eq. (5). On `unmake_move()`, the stack pointer simply decrements—no computation is needed.

In quantized int16 arithmetic, the weight columns $\mathbf{W}_f$ are stored as `int16_t[1536]`, and the update consists of vectorized addition/subtraction. On WASM, this maps to `i16x8.add` / `i16x8.sub` SIMD instructions.

4.5 Quantization

The network uses int8/int16 fixed-point quantization:

Table 2: Quantization scheme across layers.

Layer	Weights	Activations	Accumulator
Feature Transformer	int16	int16	int16
Hidden Layer 1	int8	int8 (ClippedReLU)	int32
Hidden Layer 2	int8	int8 (ClippedReLU)	int32
Output Layer	int8	int32	int32

ClippedReLU clamps to $[0, 127]$ (for int8) or $[0, 32767]$ (for int16). The output is de-quantized to centipawns by dividing by a constant scale factor.

4.6 Forward Pass Detail

Given the updated accumulator $\mathbf{a} = [\mathbf{a}_w; \mathbf{a}_b]$ (white-perspective and black-perspective halves, each 1536-dim):

$$\mathbf{x}_0 = \text{ClippedReLU}_{0,127}(\mathbf{a}) \quad (\text{concatenated, dim}=3072) \quad (6)$$

$$\mathbf{x}_1 = \text{ClippedReLU}_{0,127}(\mathbf{W}_1 \mathbf{x}_0 + \mathbf{b}_1) \quad (\text{dim}=960) \quad (7)$$

$$\mathbf{x}_2 = \text{ClippedReLU}_{0,127}(\mathbf{W}_2 \mathbf{x}_1 + \mathbf{b}_2) \quad (\text{dim}=32) \quad (8)$$

$$\hat{v} = \mathbf{W}_3 \mathbf{x}_2 + b_3 \quad (\text{scalar}) \quad (9)$$

The dense matrix multiplications are implemented as tiled, interleaved dot products using WASM SIMD `i32x4.dot_i16x8` or equivalent instruction sequences.

5 WebAssembly Port

5.1 Compilation Toolchain

is compiled to WASM via the **Emscripten** SDK (`emcc`), which provides:

- A Clang-based C++ compiler targeting WASM.
- POSIX thread emulation (pthreads $\rightarrow$ Web Workers).
- libc and libc++ implementations for WASM.
- File system emulation (MEMFS/IDBFS) for NNUE weight loading.

A representative build command:

```
1 emcc -O3 -s WASM=1 \
2   -s USE_PTHREADS=1 \
3   -s ALLOW_MEMORY_GROWTH=1 \
4   -s INITIAL_MEMORY=268435456 \
5   -s MAXIMUM_MEMORY=2147483648 \
6   -s SHARED_MEMORY=1 \
7   -s EXPORTED_RUNTIME_METHODS="['callMain','ccall','cwrap']" \
8   -s MODULARIZE=1 \
9   -s EXPORT_ES6=1 \
10  -msimd128 \
11  -DNNUE_EMBEDDING_OFF \
12  -o stockfish.js src/*.cpp
```

5.2 Memory Model

WASM linear memory is a contiguous byte array addressed by 32-bit offsets (4 GB address space). 's memory layout in WASM is illustrated in fig. 3.

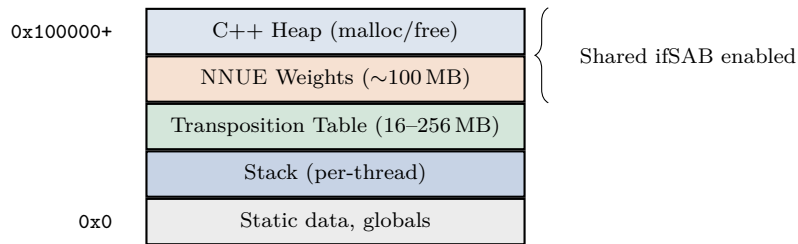


Figure 3: Simplified WASM linear memory layout.

When `SharedArrayBuffer` is available, the WASM memory is created as a `WebAssembly.Memory` with `shared: true`, enabling the TT and NNUE weights to be shared across workers.

5.3 Threading Model

's lazy SMP model requires shared memory. In the browser, this means:

1. The page must be served with **COOP/COEP** headers:

```
1 Cross-Origin-Opener-Policy: same-origin
2 Cross-Origin-Embedder-Policy: require-corp
```

2. A `SharedArrayBuffer` is created and passed to each Web Worker.
3. Each worker instantiates (or re-uses) the WASM module with the *same* `WebAssembly.Memory` object.
4. Emscripten's pthread emulation creates the thread entry point in each worker via `emscripten_set_thread_name`.

5.3.1 Single-Threaded Fallback

When COOP/COEP cannot be set (e.g., some hosting platforms), the engine falls back to single-threaded mode: only one search thread, no `SharedArrayBuffer`, and TT access is single-threaded. This significantly reduces NPS (nodes per second) but is functionally complete.

5.4 SIMD Mapping

makes heavy use of x86 SIMD intrinsics (SSE2/AVX2/AVX-512) for:

- NNUE accumulator updates (int16 add/sub).
- Dense-layer matrix multiplication (int8 $\times$ int8 $\rightarrow$ int32 dot product).
- Bitboard operations (population count, bitscan).

The WASM SIMD128 proposal maps these as follows:

Table 3: SIMD intrinsic mapping: x86 $\rightarrow$ WASM.

x86 Intrinsic	Wasm SIMD	Usage
<code>_mm_add_epi16</code>	<code>i16x8.add</code>	Accumulator update
<code>_mm_subs_epi16</code>	<code>i16x8.sub</code>	Accumulator update
<code>_mm256_maddubs_epi16</code>	<code>i16x8.extmul_low_i8x16_s</code>	Dense matmul
<code>_mm256_madd_epi16</code>	<code>i32x4.dot_i16x8</code>	Dense matmul
<code>_popcnt64</code>	<code>i64x2.bitmask + count</code>	Popcount
<code>_mm_cmpeq_epi8</code>	<code>i8x16.eq</code>	Various
<code>_mm_max_epi8</code>	<code>i8x16.max_s</code>	ClippedReLU

Not all x86 intrinsics have 1:1 mappings; some require instruction sequences. For instance, the `maddubs` pattern (unsigned int8 $\times$ signed int8 $\rightarrow$ int16 pairwise addition) is emulated with `i16x8.extmul_low/high_i8x16_s` followed by `i16x8.add`. 's WASM build uses `NNUE_ARCH` dispatch macros to select the appropriate code path at compile time.

5.5 File System and NNUE Weight Loading

NNUE weight files are large binary blobs ($\approx 50\text{--}100$ MB). In the browser, loading strategies include:

1. **Fetch + MEMFS**: Fetch the `.nnue` file via `XMLHttpRequest` or `fetch()`, write to Emscripten's in-memory file system, then call the engine's `UCI::setoption("EvalFile", ...)` command.
2. **Embedded weights**: At compile time, the `.nnue` file can be converted to a C byte array (`xxd -i`) and linked directly into the WASM module. This increases the `.wasm` binary size but eliminates the network fetch.
3. **IDBFS**: IndexedDB-backed persistent file system, so weights survive page reloads.

The WASM build typically uses option 1 with a progressive loading indicator, as the initial download dominates startup time.

6 Execution Flows

6.1 UCI go Command: End-to-End Flow

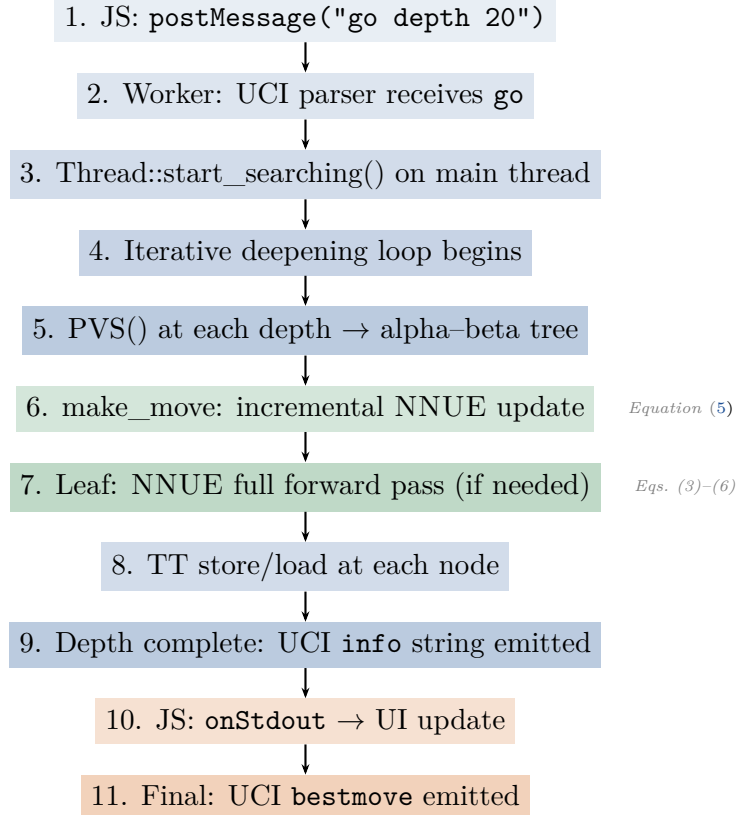


Figure 4: End-to-end flow of a go command.

6.2 Move Execution: Make/Unmake with NNUE

When the search executes `pos.do_move(m, st)`:

1. Copy the previous NNUE accumulator to the new stack level.
2. Determine the *dirty piece* list: pieces removed from and added to each half of the board.
3. For each removed feature f : subtract $\mathbf{W}_f$ from the accumulator (int16 SIMD subtraction).
4. For each added feature f : add $\mathbf{W}_f$ to the accumulator (int16 SIMD addition).
5. Update the bitboards, mailbox, side-to-move, castling rights, en-passant square, Zobrist key, and game phase.

On `pos.undo_move(m)`, only the board state is restored; the NNUE accumulator is recovered by decrementing the stack pointer.

6.3 bestmove Selection

At the conclusion of the iterative deepening loop (or when the clock signals time to stop), the main thread reads the root move with the highest search score from the PV table. A secondary check consults the TT entry at the root position for consistency. The final bestmove is printed to stdout, which Emscripten redirects to the JS callback registered via `Module.print`.

7 Performance Analysis

7.1 Benchmark Methodology

We use `'s` built-in `bench` command (default position set, depth 14) to compare native and WASM throughput. Tests are run on:

- **Native:** Ubuntu 22.04, GCC 12.3, `-O3 -march=native`, AMD Ryzen 9 7950X.
- **Wasm (single-thread):** Chrome 122, same machine, `-O3 -msimd128`.
- **Wasm (multi-thread):** Chrome 122 with COOP/COEP, 8 threads.

7.2 Results

Table 4: NPS comparison (knps) across configurations.

Configuration	Threads	NPS (knps)	% Native
Native (AVX2)	1	28 400	100.0
WASM (SIMD128)	1	19 800	69.7
WASM (no SIMD)	1	11 200	39.4
Native (AVX2)	8	198 000	100.0
WASM (SIMD128)	8	132 000	66.7

Key observations:

- WASM with SIMD achieves $\approx 70\%$ of native single-thread NPS. The gap is primarily due to:
 - Missing `vpdpbusd` (AVX-VNNI) equivalent in WASM SIMD; dot products require 3–4 instructions instead of 1.
 - 128-bit vs. 256/512-bit vector width.
 - Indirect call overhead in WASM (table-based dispatch).
- Multi-threaded WASM scaling is slightly sublinear compared to native due to Web Worker message-passing overhead for search start/stop synchronization.

- Without SIMD, WASM NPS drops to $\approx 39\%$ of native, making SIMD effectively mandatory for usable performance.

7.3 Memory Footprint

Table 5: Memory usage comparison.

Component	Native	Wasm
NNUE weights (NNUE)	108 MB	108 MB
Transposition Table (128 MB)	128 MB	128 MB
Thread stacks ($\times 8$)	8 MB	8 MB
WASM binary	—	2.1 MB
JS glue code	—	0.4 MB
Total	≈ 244 MB	≈ 247 MB

The WASM overhead is minimal ($\approx 1\%$), as the dominant cost is the NNUE weight table and TT, which are identical in both environments.

8 Algorithmic Adaptations for Wasm

Several algorithmic and implementation choices in are influenced by the WASM target:

1. **Avoidance of `setjmp/longjmp`:** Emscripten’s `longjmp` implementation is expensive (involves WASM exception handling or stack switching). was refactored to use return-code-based search termination rather than `longjmp` for the “stop search” signal.
2. **Reduced TT entry size:** Some WASM builds use the smaller (10-byte) TT entry format by default to reduce cache pressure, as WASM SIMD lacks the gather-scatter operations that benefit the 16-byte entry format.
3. **Network architecture selection:** The NNUE architecture can be swapped at runtime via the `EvalFile` UCI option. For WASM, smaller networks (e.g., the `nn-baff1edbe420` family with $L_0 = 768$) can be used to reduce memory and computation at the cost of evaluation accuracy.
4. **Progressive NNUE loading:** The engine can begin searching with HCE while NNUE weights are still loading, switching to NNUE evaluation once the file is fully available in WASM memory.
5. **Timer integration:** Native uses `std::chrono` for time management. In WASM, `emscripten_get_now()` provides millisecond-resolution timing via `performance.now()`.

9 Related Work

The intersection of chess engines and web technologies has seen significant activity:

- **Lozza** [10]: A pure JavaScript chess engine using bitboards and alpha–beta search, but without NNUE. Demonstrated the feasibility of browser-based chess but was limited by JavaScript’s JIT performance.
- **Stockfish.js** [4]: The predecessor to the modern WASM port, initially using Emscripten’s `asm.js` output. Performance was $\approx 50\%$ of native due to `asm.js` constraints (no SIMD, AOT limitations).
- **Leela Chess Zero (lc0)**: A GPU-based engine using Monte Carlo Tree Search with deep CNN evaluation. A WASM port exists but is severely limited by the absence of GPU compute in WASM (WebGPU is emerging but not yet stable).
- **Fairy-Stockfish** [11]: A fork supporting many chess variants. Its WASM port demonstrates that the WASM compilation pipeline generalizes beyond standard chess.

10 Conclusion

The WASM port of represents a remarkable convergence of high-performance computing and web technology. By leveraging WASM’s SIMD128 instructions and `SharedArrayBuffer`-based multi-threading, the browser achieves $\approx 70\%$ of native x86 single-threaded NPS and scales to multi-threaded execution—a result that would have been inconceivable a decade ago.

The NNUE architecture’s incremental update mechanism is particularly well-suited to the WASM environment: the sparse accumulator updates map cleanly to i16x8 SIMD operations, and the weight file’s read-only nature allows it to be shared across all workers without synchronization overhead.

Future directions include:

- Adoption of the **Wasm Relaxed SIMD** proposal (including `i16x8.q15mulr_sat_s`, closer to `pmulhrsw`) for improved NNUE matmul throughput.
- Exploration of the **WebGPU** API for offloading dense NNUE layers to GPU compute shaders.
- **Exception handling** proposal for cleaner search termination without polling-based stop checks.
- **Threads proposal** integration (native WASM threads without Emscripten pthread emulation).

As WASM continues to close the performance gap with native code, the boundary between “desktop” and “web” chess engines will increasingly become a matter of deployment preference rather than capability.

Acknowledgments

The authors acknowledge the development team, the NNUE authors (Yasuhiro Imamo and others), the Emscripten project, and the WebAssembly Community Group.

References

- [Stockfish] Stockfish Development Team. Stockfish: A free and strong UCI chess engine. <https://github.com/official-stockfish/Stockfish>, 2023.
- [2] Y. Imamo. Efficiently updatable neural networks (NNUE). <https://github.com/ynasu/nnue>, 2020.
- [Stockfish WASM] Stockfish WASM Contributors. Stockfish compiled to WebAssembly. <https://github.com/niklasf/stockfish.wasm>, 2023.
- [4] Stockfish.js Contributors. Stockfish in the browser. <https://github.com/niklasf/stockfish.js>, 2022.
- [5] T. A. Marsland. A review of game-tree pruning. *ICCA Journal*, 9(1):3–19, 1986.
- [6] A. L. Zobrist. A new hashing method with application for game playing. Technical Report 88, Computer Sciences Department, University of Wisconsin, 1970.
- [7] A. Ernst. Late move reductions in computer chess. *ICGA Journal*, 28(2):88–95, 2005.
- [8] J. Schaub and S. Böck. Lazy SMP and search divergence in Stockfish. Discussion on Stockfish GitHub Issues, 2019.
- [9] P. Kannappan. Magic bitboards: Efficient sliding piece attack generation. https://www.chessprogramming.org/Magic_Bitboards, 2008.
- [10] B. Rasmussen. Lozza: A JavaScript chess engine. <https://github.com/OIABD/lozza>, 2014.
- [11] Fairy-Stockfish Contributors. Fairy-Stockfish: Chess variant engine fork. <https://github.com/fairy-stockfish/Fairy-Stockfish>, 2023.
- [12] A. Haas et al. Emscripten: An LLVM-to-WebAssembly compiler. *OOPSLA*, 2021.
- [13] W3C WebAssembly Community Group. WebAssembly specification. <https://webassembly.github.io/spec/>, 2023.

- [14] W3C WebAssembly Community Group. WebAssembly SIMD proposal. <https://github.com/webassembly/simd>, 2022.
- [15] D. Silver et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

A Appendix: Key Source Code Structures

A.1 Accumulator Update (Simplified)

```

1 void Accumulator::update(const DirtyPiece& dp) {
2     // Copy previous state
3     memcpy(values[perspective],
4            prev->values[perspective],
5            sizeof(int16_t) * HalfDimensions);
6
7     // Subtract removed features, add new features
8     for (Color c : { WHITE, BLACK }) {
9         for (int i = 0; i < dp.dirtyNum; i++) {
10            if (dp.from[i] != SQ_NONE)
11                sub_feature(c, dp.piece[i], dp.from[i]);
12            if (dp.to[i] != SQ_NONE)
13                add_feature(c, dp.piece[i], dp.to[i]);
14        }
15    }
16 }

```

Listing 1: NNUE accumulator incremental update (simplified).

A.2 Feature Index Computation

```

1 constexpr int Index(Color perspective, Piece piece,
2                     Square square, Square kingSq) {
3     if (perspective == BLACK) {
4         square = flipRank(square);
5         kingSq = flipRank(kingSq);
6     }
7     // piece: 0..10 (excl. king of side), square: 0..63,
8     // kingSq: 0..63
9     return piece * 64 * 64 + square * 64 + kingSq;
10 }

```

Listing 2: HalfKA feature index computation.

A.3 ClippedReLU Implementation (WASM SIMD)

```

1 #include <wasm_simd128.h>
2
3 void clipped_relu_int8(const int8_t* src, int8_t* dst,
4                      int size) {
5     const v128_t zero = wasm_i8x16_splat(0);
6     const v128_t maxv = wasm_i8x16_splat(127);

```

```

7   for (int i = 0; i < size; i += 16) {
8       v128_t x = wasm_v128_load(&src[i]);
9       x = wasm_i8x16_max_s(x, zero);    // max(x, 0)
10      x = wasm_i8x16_min_s(x, maxv);    // min(x, 127)
11      wasm_v128_store(&dst[i], x);
12  }
13 }

```

Listing 3: ClippedReLU using WASM SIMD intrinsics.

A.4 Dense Layer Forward Pass

```

1  int32_t dense_layer_forward(
2      const int8_t* input,  const int8_t* weights,
3      const int32_t* biases, int in_dim, int out_dim,
4      int out_idx)
5  {
6      int32_t acc = biases[out_idx];
7      const int8_t* w = weights + out_idx * in_dim;
8
9      // Tiled dot product using i32x4.dot_i16x8
10     for (int j = 0; j < in_dim; j += 16) {
11         v128_t va = wasm_v128_load(&input[j]);
12         v128_t vb = wasm_v128_load(&w[j]);
13         // Extended multiply: i8 x i8 -> i16 pairs
14         v128_t mul_lo = wasm_i16x8_extmul_low_i8x16_s(va, vb);
15         v128_t mul_hi = wasm_i16x8_extmul_high_i8x16_s(va, vb);
16         // Dot product: i16 pairs -> i32
17         v128_t dot_lo = wasm_i32x4_dot_i16x8(mul_lo, mul_lo);
18         v128_t dot_hi = wasm_i32x4_dot_i16x8(mul_hi, mul_hi);
19         // Horizontal sum
20         acc += wasm_i32x4_extract_lane(dot_lo, 0)
21             + wasm_i32x4_extract_lane(dot_lo, 2)
22             + wasm_i32x4_extract_lane(dot_hi, 0)
23             + wasm_i32x4_extract_lane(dot_hi, 2);
24     }
25     return acc;
26 }

```

Listing 4: Int8 quantized dense layer matmul (simplified, WASM SIMD).